

Національний технічний університет України «Київський
політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Лабораторна робота № 6

з дисципліни «Розробка мобільних
застосунків під Android»

Тема: «Змодельовати тему дипломного проєкту шляхом
самостійного формування завдання»

Виконав:

студент групи ІА-31

Саkun Дмитро

Перевірів:

Орленко С.П

Київ 2026

Мета роботи: змодельовати головну проблему наступного року навчання (у вигляді вибору теми дипломного проекту) шляхом самостійного формування завдання на 6 лабораторну роботу.

ЗАВДАННЯ

Написати програму під платформу Андроїд, яка буде заснована на темі, яка не розглядалась на попередніх лабораторних роботах (попередні навички теж можна використовувати, але «основою» має бути саме нова обрана тема).

Примітка: Очевидно, що конкретного варіанту не передбачено, студент сам формує завдання та вигляд програми. Наприклад, на попередніх лабах не була приділена увага web-у, створенню віджетів, роботі з зовнішнім обладнанням, створенню ігрових застосунків і т.д.

Хід роботи

Для виконання роботи було обрано тему «Взаємодія мобільного додатку з Web API та асинхронний парсинг структури JSON». Було розроблено фінансовий застосунок «Crypto Price Tracker», який відправляє HTTP-запити на сервер CoinGecko REST API, отримує актуальні курси цифрових активів (Bitcoin, Ethereum, Solana) і виводить інформацію користувачеві. Оскільки мережеві HTTP-запити за стандартами ОС Android заблоковані у головному (UI) потоці для запобігання зависання інтерфейсу, у додатку застосовано механізм Kotlin Coroutines із використанням фонового контексту Dispatchers.IO.

Посилання на повний вихідний код проєкту (GitHub):

<https://github.com/verom1/SensorLab5>

Лістинг основного коду (MainActivity.kt):

```
package com.example.cryptolab_final

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.Refresh
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
```

```

import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.launch
import kotlinx.coroutines.withContext
import org.json.JSONObject
import java.net.URL

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            MaterialTheme {
                Surface(
                    modifier = Modifier.fillMaxSize(),
                    color = MaterialTheme.colorScheme.background
                ) {
                    CryptoTrackerScreen()
                }
            }
        }
    }
}

@Composable
fun CryptoTrackerScreen() {
    var btcPrice by remember { mutableStateOf("---") }
    var ethPrice by remember { mutableStateOf("---") }
    var solPrice by remember { mutableStateOf("---") }

    var isLoading by remember { mutableStateOf(false) }
    var errorMessage by remember { mutableStateOf("") }

    val scope = rememberCoroutineScope()

    fun fetchPrices() {
        scope.launch {
            isLoading = true
            errorMessage = ""
            try {
                val response = withContext(Dispatchers.IO) {
                    URL("https://api.coingecko.com/api/v3/simple/price?ids=bitcoin,ethereum,solana&vs_currencies=usd").readText()
                }

                val json = JSONObject(response)
                btcPrice = "$ ${json.getJSONObject("bitcoin").getString("usd")}"
                ethPrice = "$ ${json.getJSONObject("ethereum").getString("usd")}"
                solPrice = "$ ${json.getJSONObject("solana").getString("usd")}"
            } catch (e: Exception) {
                errorMessage = "Помилка мережі. Перевірте підключення."
            } finally {
                isLoading = false
            }
        }
    }

    LaunchedEffect(Unit) {
        fetchPrices()
    }

    Column(
        modifier = Modifier
            .fillMaxSize()
            .padding(24.dp),
    )

```

```

        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        Text(text = "Crypto Tracker", fontSize = 32.sp, fontWeight = FontWeight.Bold)
        Text(text = "Live Web API: CoinGecko", color =
MaterialTheme.colorScheme.secondary)
        Spacer(modifier = Modifier.height(40.dp))

        CryptoCard("Bitcoin (BTC)", btcPrice)
        Spacer(modifier = Modifier.height(16.dp))
        CryptoCard("Ethereum (ETH)", ethPrice)
        Spacer(modifier = Modifier.height(16.dp))
        CryptoCard("Solana (SOL)", solPrice)

        Spacer(modifier = Modifier.weight(1f))

        if (isLoading) {
            CircularProgressIndicator()
        } else if (errorMessage.isNotEmpty()) {
            Text(text = errorMessage, color = MaterialTheme.colorScheme.error)
            Spacer(modifier = Modifier.height(8.dp))
            Button(onClick = { fetchPrices() }) {
                Text("Спробувати ще раз")
            }
        } else {
            Button(
                onClick = { fetchPrices() },
                modifier = Modifier.fillMaxWidth().height(50.dp)
            ) {
                Icon(Icons.Default.Refresh, contentDescription = "Оновити")
                Spacer(Modifier.width(8.dp))
                Text("Оновити курси", fontSize = 18.sp)
            }
        }
        Spacer(modifier = Modifier.height(16.dp))
    }
}

@Composable
fun CryptoCard(name: String, price: String) {
    Card(
        modifier = Modifier.fillMaxWidth(),
        shape = RoundedCornerShape(16.dp),
        elevation = CardDefaults.cardElevation(defaultElevation = 6.dp)
    ) {
        Row(
            modifier = Modifier
                .fillMaxWidth()
                .padding(20.dp),
            horizontalArrangement = Arrangement.SpaceBetween,
            verticalAlignment = Alignment.CenterVertically
        ) {
            Text(text = name, fontSize = 20.sp, fontWeight = FontWeight.SemiBold)
            Text(
                text = price,
                fontSize = 20.sp,
                fontWeight = FontWeight.Bold,
                color = MaterialTheme.colorScheme.primary
            )
        }
    }
}

```

Результати роботи

Додаток успішно скомпільовано у файл app-debug.apk та протестовано через середовище хмарного емулятора Appetize.io з активним мережевим з'єднанням. Інтерфейс коректно відмальовує отримані з сервера дані.

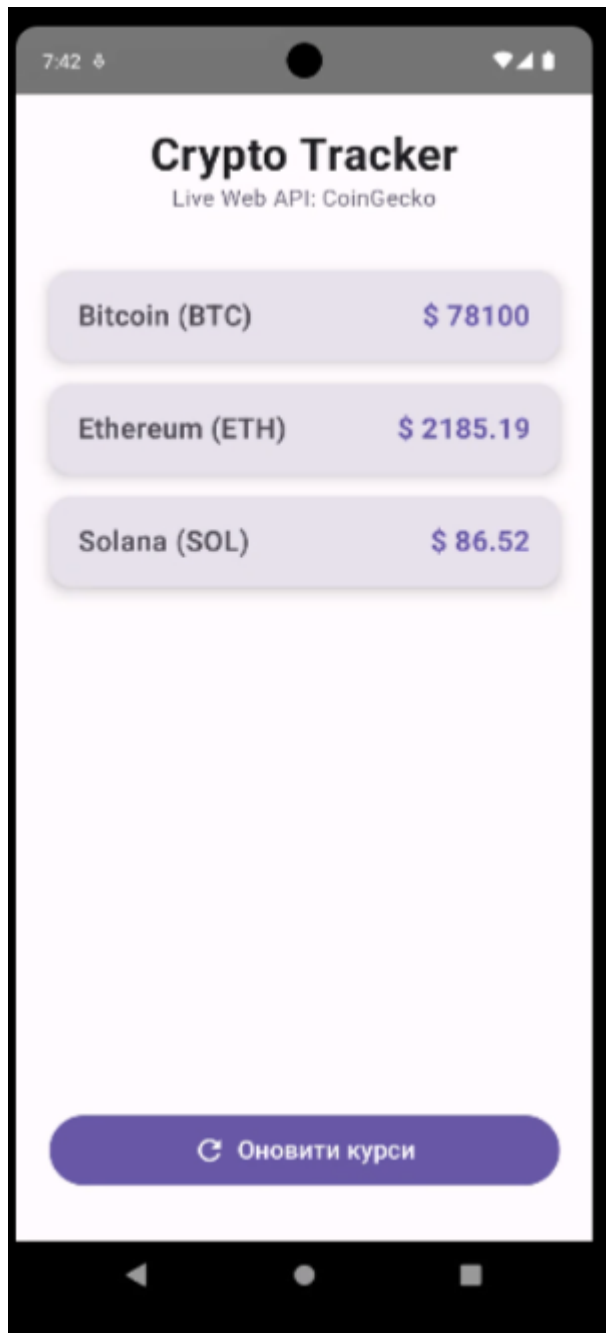


Рис. 1. Головний екран працюючого застосунку «Crypto Price Tracker»

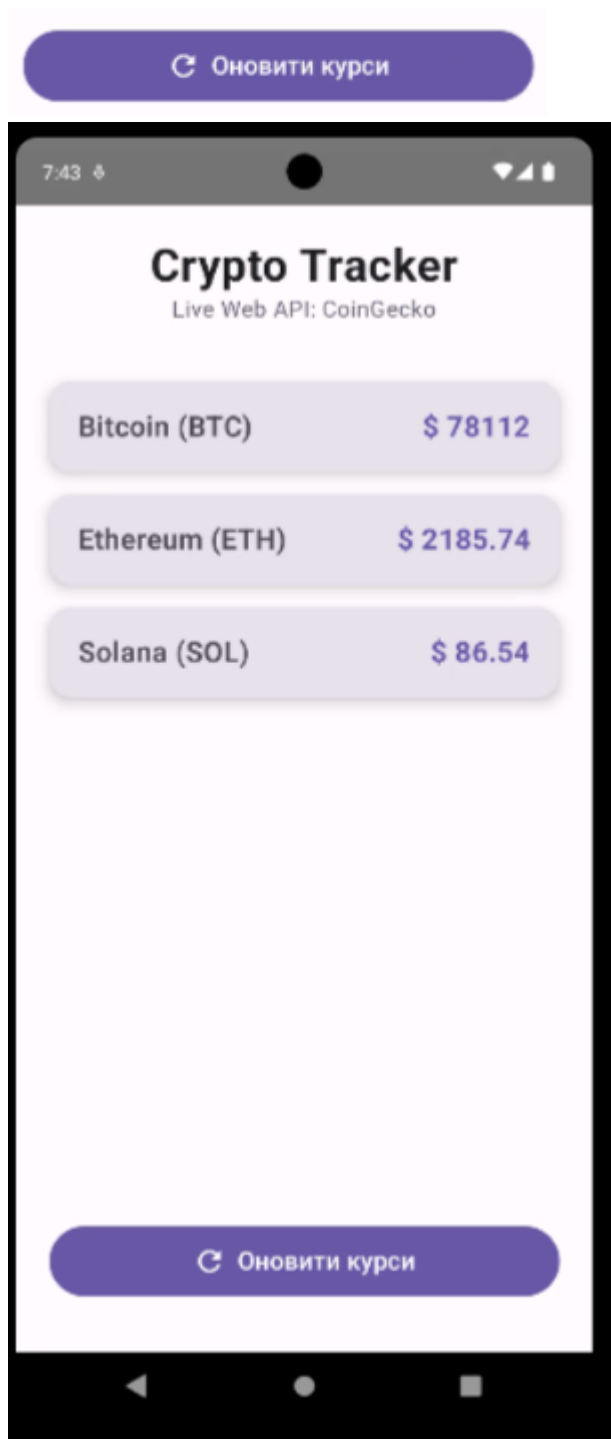


Рис. 2. Процес асинхронного запиту та завантаження актуальних даних при натисканні кнопки оновлення

Висновки

Під час виконання лабораторної роботи №6 на вільну тему було успішно змодельовано практичну задачу клієнт-серверної взаємодії. Я детально ознайомився із принципами побудови HTTP-запитів до REST API за допомогою стандартних інструментів мови Kotlin без перевантаження проекту громіздкими сторонніми

бібліотеками.

Було реалізовано коректне архітектурне розділення потоків: важкі операції зчитування тексту з мережевої URL-адреси та парсинг структури JSON виконуються асинхронно у фоновому пулі потоків Dispatchers.IO, що виключає ризик блокування основного графічного інтерфейсу (UI thread). Динамічне відображення даних забезпечено реактивним станом фреймворку Jetpack Compose. Проєкт завантажено до відкритого репозиторію GitHub.